

컴퓨터비전_인공지능(AI)개론

10. 다중분류2

- 문제 정의하기
- 신경망과 딥러닝
- 활성화 함수와 ReLU 함수
- GPU 사용하기
- 데이터 전처리
- 미니 배치 학습법
- 데이터 준비 (Dataset, Transforms, 데이터로더)
- 모델 정의
- 경사 하강법
- 결과 확인
- 은닉층 추가하기

지금까지 우리는 모델에게 '숫자' 데이터를 다루는 법을 가르쳤습니다.
마치 계산기의 숫자 버튼을 누르면 정해진 답이 나오는 것처럼, 명확한 값들을 입력으로 주었죠.

한 단계 더 나아가, 모델이 직접 '보고 이해하는' 법을 배우게 됩니다.
바로 손으로 쓴 숫자 이미지를 판별하는 것입니다.

우리는 숫자를 '784'라는 값으로 인지하지 않고, 그 '모양'으로 인지합니다.

이처럼 컴퓨터가 시각적인 정보를 이해하게 하는 기술은 자율주행 자동차가 사물을 인식하고,
의료 AI가 질병을 진단하는 핵심 원리입니다.

픽셀과 데이터 표현

사람은 '7'이라는 이미지를 보고 직관적으로 숫자 7임을 압니다.

하지만 컴퓨터에게 이미지는 숫자로 가득 찬 거대한 격자판일 뿐입니다.

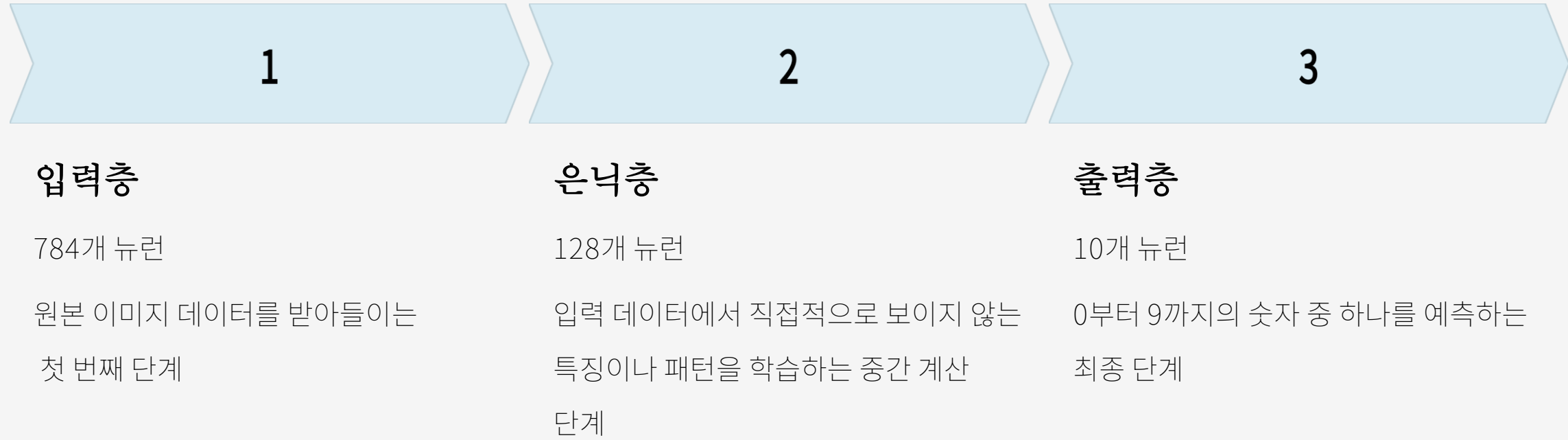
MNIST 이미지는 가로 28개, 세로 28개의 픽셀로 이루어져 있습니다 (총 $28 \times 28 = 784$ 개).

각 픽셀은 0(검은색)부터 255(흰색)까지의 숫자로 밝기를 표현합니다.

벡터화 과정

파이토치로 이 데이터를 불러오면 $[0, 1]$ 범위의 소수점으로 정규화되고, [채널, 높이, 너비] 형태, 즉 $[1, 28, 28]$ 모양의 3차원 텐서가 됩니다.

'완전결합 신경망'은 1차원 데이터 입력을 필요로 합니다. 따라서 28×28 크기의 2차원 이미지 데이터를 784개의 숫자가 늘어선 1차원 벡터로 쭉 펴주는 전처리 과정이 필요합니다.



예를 들어, 숫자 '8'을 인식할 때, 은닉층은 '상단 동그라미'와 '하단 동그라미'라는 중간 특징을 학습함
이렇게 여러 층을 쌓아 복잡한 문제를 해결하는 것이 딥러닝의 핵심입니다.

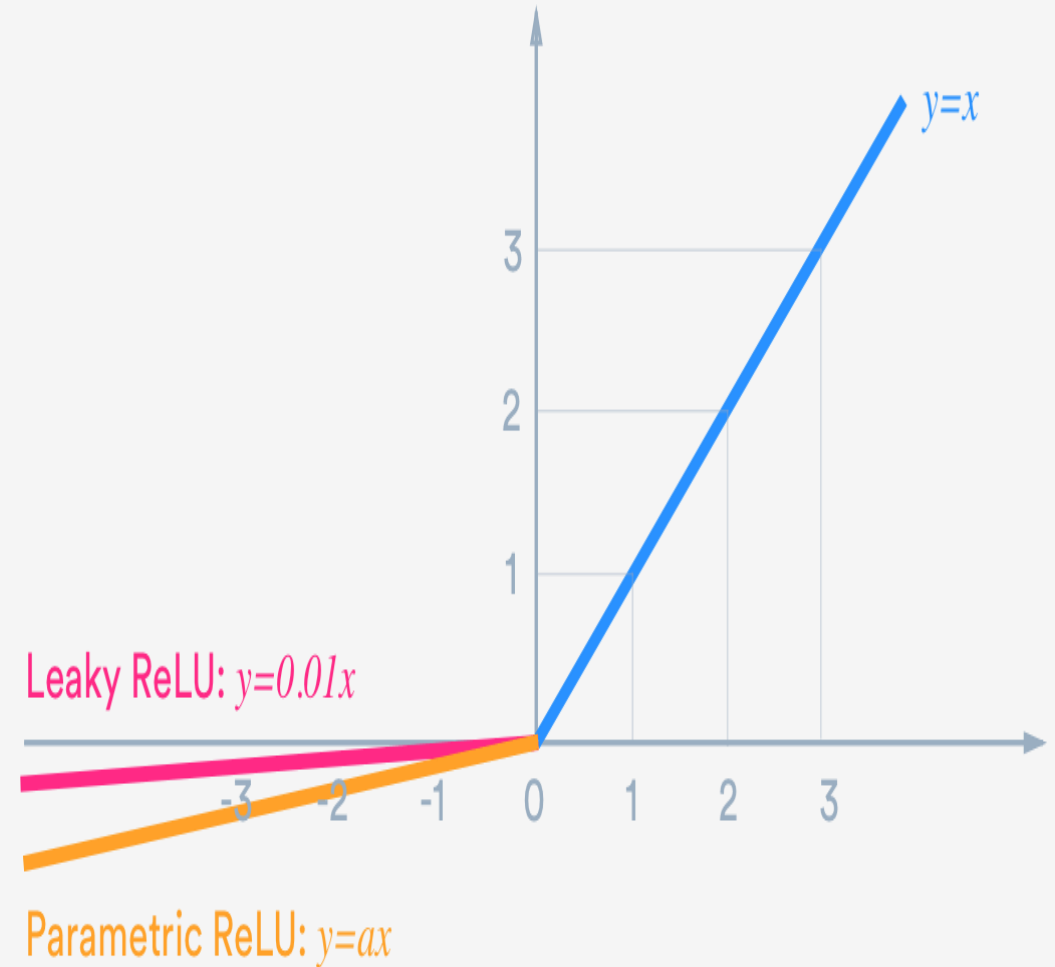
비선형성의 필요성

단순히 선형 함수만 여러 겹 쌓는 것은 수학적으로는 결국 하나의 큰 선형 함수와 같습니다. 층을 깊게 쌓는 의미를 가지려면 각 층의 결과에 '비선형 함수'를 적용해야 합니다. 이 역할을 하는 것이 바로 **활성화 함수(Activation Function)**입니다.

ReLU의 작동 원리

ReLU (Rectified Linear Unit)는 입력값이 0보다 작으면 0을 출력하고, 0보다 크면 입력값을 그대로 출력합니다.

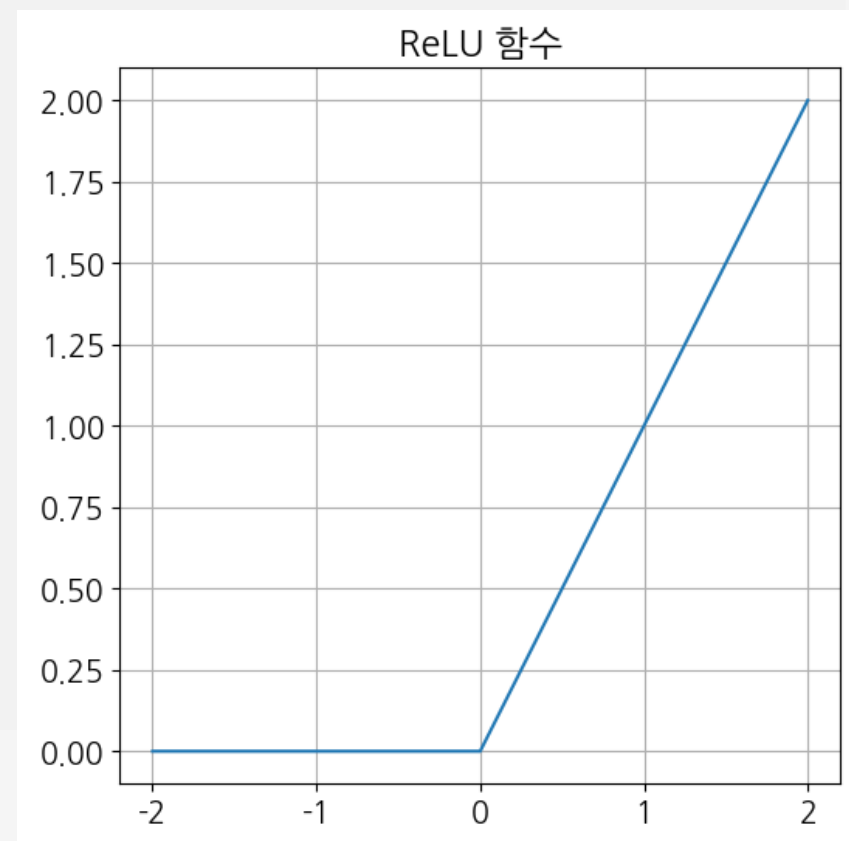
ReLU는 '신경세포의 스위치'와 같습니다. 특정 자극이 임계점(0)을 넘으면 신호를 전달하고, 넘지 못하면 신호를 차단합니다.



ReLU 함수가 어떤 모양을 가졌는지 그래프로 확인해보겠습니다. 그래프에서 볼 수 있듯이, 입력값(x축)이 0보다 작을 때는 출력이 모두 0이고, 0보다 클 때는 입력값이 그대로 출력되는 직선

```
relu = nn.ReLU() # ReLU 함수 인스턴스 생성  
x_np = np.arange(-2.0, 2.1, 0.25)  
x = torch.tensor(x_np).float()  
y = relu(x)
```

이 지점에서 꺾이는 '비선형' 특성이
딥러닝 모델에 깊이를 더해주는 핵심입니다.



GPU의 역할

MNIST 데이터는 훈련용으로만 6만 장에 달합니다.
이 방대한 양의 연산을 CPU로 처리하기엔 시간이 너무 오래 걸립니다.

GPU는 수많은 계산을 동시에 처리하는 '**병렬 연산**'에 특화되어 있어 딥러닝 학습 속도를 폭발적으로 향상시켜 줍니다.

미니 배치 학습법

6만 개의 데이터를 한꺼번에 모델에 보여주고 학습시키는 것은 매우 비효율적입니다.

대신, 데이터를 작은 묶음(예: 100개씩)으로 나누어 학습을 여러 번 반복합니다. 이 작은 묶음 하나를 '**미니 배치**'라고 부릅니다.



파이토치에서는 코드를 통해 어떤 장치(CPU or GPU)에서 계산을 수행할지 명시적으로 지정해야 합니다.

디바이스 확인

```
device = torch.device("cuda:0"
                       if torch.cuda.is_available()
                       else "cpu")
```

`torch.cuda.is_available()`은 현재 환경에서 CUDA(NVIDIA GPU)를 사용할 수 있는지 확인하는 함수입니다.

이 결과에 따라 `device` 변수에 'cuda:0'(첫 번째 GPU) 또는 'cpu'라는 문자열을 할당합니다.

실행 결과

GPU 사용 가능 시:

```
cuda:0
```

GPU 사용 불가 시:

```
cpu
```

이 `device` 객체는 앞으로 텐서와 모델을 어디에 위치시킬지 알려주는 주소표와 같습니다.

파이토치에서는 모든 텐서와 모델이 자신이 어느 장치(CPU 또는 GPU)에 속해있는지 정보를 가집니다. 서로 다른 장치에 있는 텐서끼리는 연산할 수 없다는 중요한 규칙이 있습니다.

01

기본 CPU 연산

특별히 장치를 지정하지 않으면 모든 텐서는 기본적으로 CPU에 생성됩니다.

03

서로 다른 디바이스 간 연산

서로 다른 곳에 위치한 데이터 간의 연산은 허용되지 않으므로, 파이토치는 RuntimeError를 발생시킵니다.

02

텐서를 GPU로

`.to(device)` 함수를 사용해 텐서를 원하는 장치로 보낼 수 있습니다.

04

양쪽 모두 GPU로 이동

문제를 해결하려면 모든 텐서를 같은 장치로 보내주면 됩니다.

파이토치에는 데이터 준비 도구인 Dataset, Transforms, DataLoader가 있습니다.
이 도구들은 마치 전문 셰프의 주방 도구와 같습니다.



Dataset

창고에서 원재료(이미지 파일)를 가져오는 역할.
원본 데이터 파일들의 복잡한 구조를 감추고, 파이썬의 리스트처럼 인덱스를 통해 간단하게 접근할 수 있는 인터페이스를 제공합니다.



Transforms

깨끗하게 씻고 먹기 좋게 다듬는 역할.
이미지 데이터를 파이토치 텐서로 바꾸고, 값의 범위를 조정하며, 모양을 바꾸는 등 다양한 가공을 수행합니다.



DataLoader

1인분씩 정확하게 소분하는 역할.
전체 데이터를 지정된 batch_size만큼씩 묶어 학습에 사용할 작은 묶음인 '미니 배치'를 자동으로 만들어줍니다.

1

ToTensor()

PIL 이미지나 Numpy 배열 같은 데이터를 파이토치 텐서로 변환합니다. 이 과정에서 픽셀 값의 범위를 [0, 255]에서 [0, 1]로 자동으로 조정합니다.

2

Normalize(mean, std)

텐서의 값 분포를 평균과 표준편차를 이용해 정규화합니다.

예를 들어, Normalize(0.5, 0.5)를 적용하면 [0, 1] 범위의 값이 [-1, 1] 범위로 변환됩니다.

3

Lambda(lambda x: ...)

미리 정의된 기능 외에, view(-1)처럼 사용자가 원하는 간단한 변환을 직접 만들어 파이프라인에 추가할 수 있게 해줍니다.

4

Compose([...])

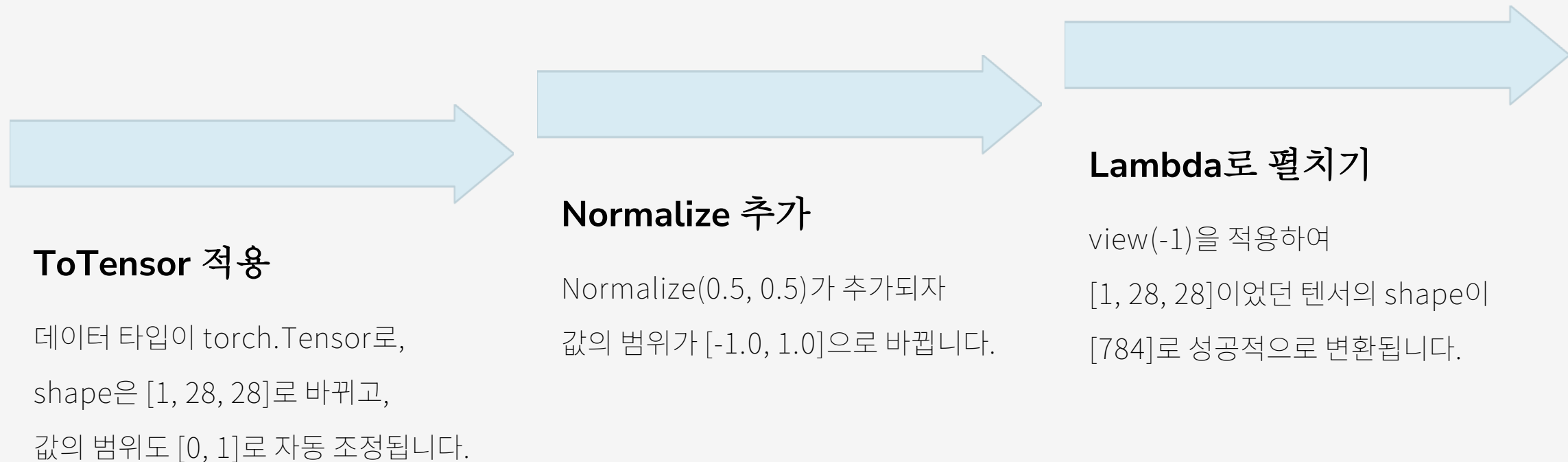
여러 단계의 변환 작업을 리스트로 묶어 순서대로 실행되는 하나의 작업 흐름으로 만들어 줍니다.

1단계: Dataset으로 원본 데이터 불러오기

torchvision을 사용해 MNIST 데이터를 다운로드하고 내용물을 확인합니다.

데이터 건수는 60,000개이며, 처음 불러온 데이터는 PIL 라이브러리의 이미지 형식입니다.

2단계: Transforms로 데이터 전처리하기



DataLoader 설정

```
batch_size = 500
# 훈련용 데이터로더
train_loader = DataLoader(
    train_set, batch_size = batch_size,
    shuffle = True)

# 검증용 데이터로더
test_loader = DataLoader(
    test_set, batch_size = batch_size,
    shuffle = False)
```

결과 확인

총 미니 배치 개수: 120

$(60,000 / 500 = 120)$

images의 shape: [500, 784]

labels의 shape: [500]

매 학습 주기마다 데이터의 순서를 무작위로
섞어줍니다(shuffle=True).

이는 모델이 데이터의 순서를 외우는 것을 방지하고
일반화 성능을 높입니다.

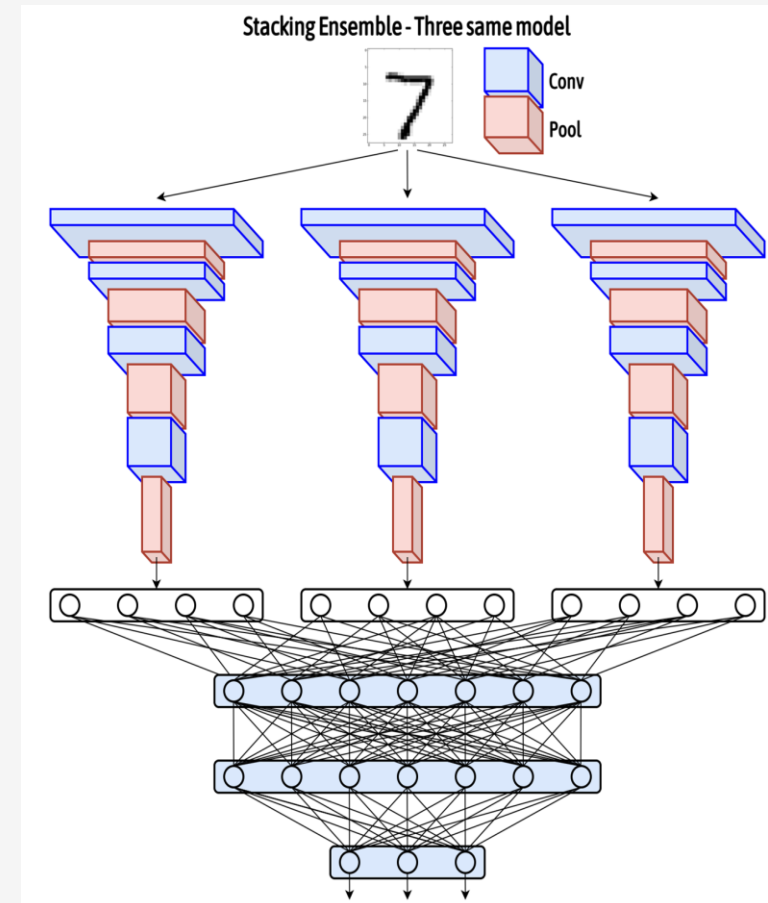
```
class Net(nn.Module):
    def __init__(self, n_input, n_output, n_hidden):
        super().__init__()
        self.l1 = nn.Linear(n_input, n_hidden)
        self.l2 = nn.Linear(n_hidden, n_output)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        x1 = self.l1(x)
        x2 = self.relu(x1)
        x3 = self.l2(x2)
        return x3
```

Net 클래스는 `__init__`에서 두 개의 `nn.Linear` 층과 `nn.ReLU`를 정의합니다.

`forward` 함수에서는 데이터가 `l1` → `relu` → `l2` 순서로 흐르도록 정의합니다.

모델 인스턴스를 생성한 후 `.to(device)`를 통해 모델의 모든 파라미터를 GPU로 보냅니다.



60K

훈련 데이터

MNIST 손글씨 숫자 이미지

100

학습 반복

횟수
Epoch 수

94.95%

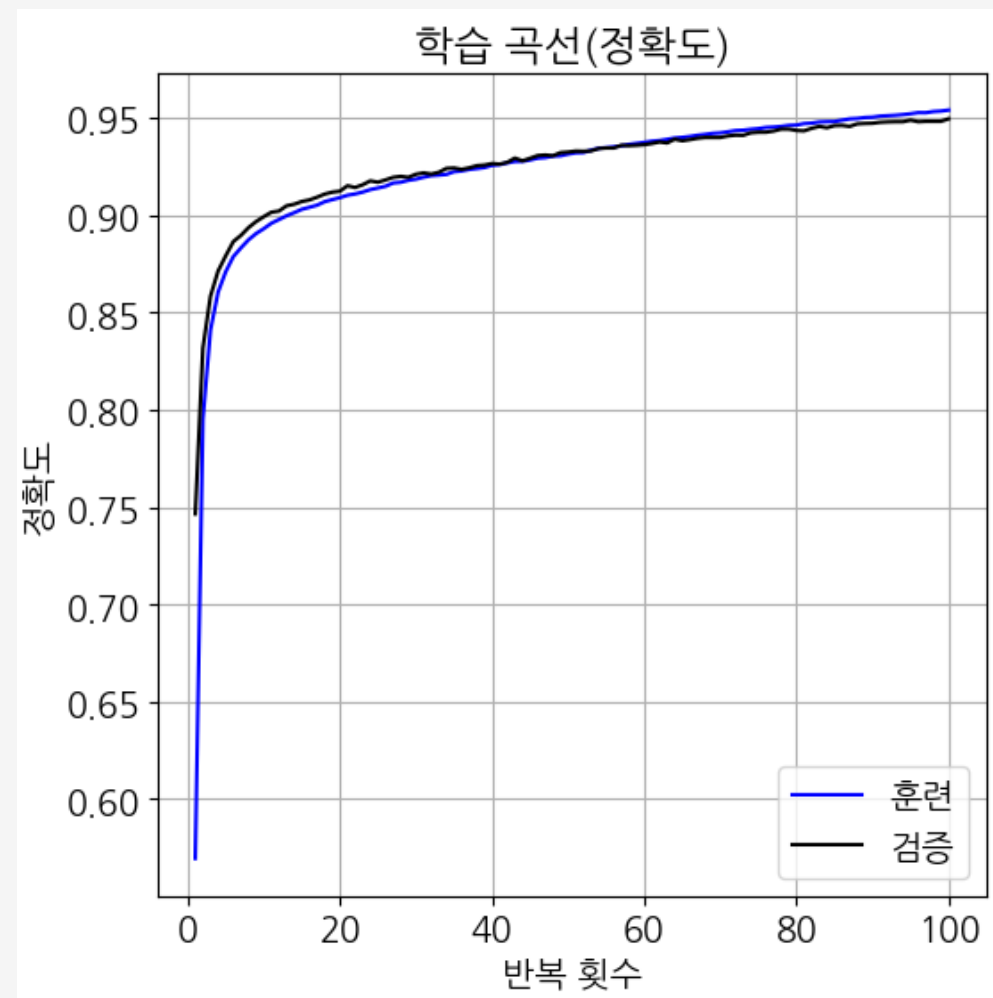
최종 정확도

검증 데이터 기준

은닉층 추가 시 성능 향상

은닉층을 하나 더 추가한 2개 은닉층 모델로 학습을 진행한 결과,
정확도가 95.81%로 이전 모델(94.95%)보다 약 1%p 향상되었습니다.

이처럼 파이토치를 사용하면 모델의 구조를 유연하게 변경하며
다양한 실험을 쉽게 수행할 수 있습니다.





경사 소실 문제와 ReLU

시그모이드 함수는 미분값이 최대 0.25를 넘지 못해 층이 깊어지면 경사가 0에 가깝게 수렴합니다. ReLU는 입력값이 양수일 경우 미분값이 항상 1이어서 경사 소실 문제를 해결합니다.



람다 표현식

람다 표현식은 작고 간단한 '이름 없는 함수'를 만들 때 사용됩니다. 함수를 정의하는 동시에 다른 함수의 인자로 바로 전달할 수 있어 코드가 훨씬 간결해집니다.



배치 사이즈의 중요성

배치 사이즈가 작을수록 더 높은 최종 정확도에 도달합니다. 작은 배치 사이즈의 '노이즈'가 모델이 국소 최적점에서 탈출하여 더 좋은 지점을 찾도록 돕는 긍정적인 역할을 합니다.

경사 소실 문제

딥러닝 모델이 깊어질수록 학습 신호인 '경사(Gradient)'가 점차 약해져,
초반 층까지 제대로 전달되지 않아 가중치 업데이트가 어려워지는 현상입니다.

특히 시그모이드 함수는 미분값이 최대 0.25에 불과해 이 문제가 심화됩니다.

경사 값 비교: 시그모이드 vs ReLU

활성화 함수를 시그모이드로 설정한 2개 은닉층 모델의 첫 번째 은닉층 가중치 경사 값을 비교합니다.

```
def forward(self, x):
    x1 = self.l1(x)
    x2 = self.sigmoid(x1)
    x3 = self.l2(x2)
    x4 = self.sigmoid(x3)
    x5 = self.l3(x4)
    return x5

# ... 중간 코드 생략

loss.backward()

w = net.to('cpu').l1.weight.grad.numpy()
```

동일 조건에서 ReLU 함수 사용 시 경사 값 평균은 **0.000848**로,
시그모이드보다 약 5배 가량 큰 값을 보입니다.
이 차이는 층이 깊어질수록 학습 성능에 지대한 영향을 미칩니다.

시그모이드 경사 값 평균: **0.000175**

ReLU 함수의 해결책

ReLU (Rectified Linear Unit)는 입력값이 양수일 때 미분값이 항상 1이므로,
경사 값이 줄어들지 않고 초반 층까지 효율적으로 전달됩니다.

이 덕분에 모델이 깊어져도 안정적인 학습이 가능해 딥러닝 발전의 핵심 동력이 되었습니다.

람다 표현식은 파이썬의 문법 중 하나로, 작고 간단한 '이름 없는 함수'를 만들 때 사용됩니다. 코드를 훨씬 간결하게 만들 수 있습니다.

왜 람다를 사용할까?

어떤 함수가 코드의 딱 한 부분에서만, 그것도 다른 함수의 인자로 전달하기 위해 잠깐 필요한 경우가 있습니다.

이때 `def`로 정식 함수를 정의하는 번거로움 없이, 함수를 정의함과 동시에 다른 함수의 인자로 바로 전달하여 코드를 간결하게 만듭니다.

특히 `transforms.Compose` 안에서 `view`를 이용해 텐서의 모양을 바꾸는 것처럼, 간단한 처리를 위해 임시 함수가 필요할 때 아주 유용합니다.

람다 표현식 사용법

일반적인 함수

```
def f(x):
    return (2 * x**2 + 2)
x = np.arange(-2, 2.1, 0.25)
y = f(x)
```

람다 표현식 정의

```
g = lambda x: 2 * x**2 + 2
y = g(x)
```

두 방식의 결과는 동일하며, 람다가 훨씬 간결한 것을 확인할 수 있습니다.

`torchvision.transforms`에서의 활용

```
transform3 = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(0.5, 0.5),
    transforms.Lambda(lambda x: x.view(-1)),
])
```

배치 사이즈는 딥러닝 모델 학습 과정에서 매우 중요한 하이퍼파라미터입니다.

한 번의 가중치 업데이트에 사용되는 데이터 샘플의 수를 결정하며, 모델의 수렴 속도와 최종 성능에 큰 영향을 미칩니다.

큰 배치 사이즈의 특징

- 파라미터 업데이트가 더 안정적
- 학습 속도는 빠를 수 있음
- 국소 최적점(Local Minimum)에 빠져 최상의 성능을 내지 못할 위험

작은 배치 사이즈의 장점

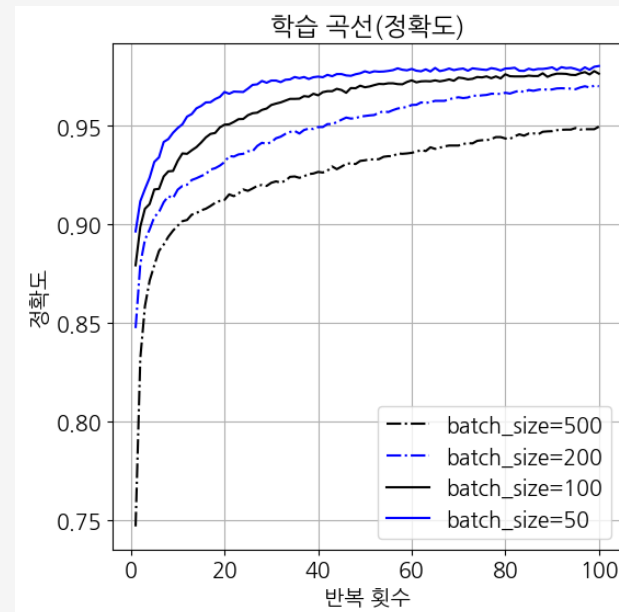
- 업데이트 과정에 '노이즈' 증가
- 이 노이즈가 국소 최적점에서 탈출하도록 도움
- 결과적으로 더 높은 최종 정확도 달성 가능

특히 학습 데이터가 많을수록 배치 사이즈 튜닝의 효과는 더욱 커지므로, 대규모 데이터셋으로 모델을 학습시킬 때는 항상 이 점을 고려해야 합니다.

배치 사이즈에 따른 정확도 변화

그래프는 다양한 배치 사이즈로 학습했을 때 검증 데이터셋의 정확도 변화를 시각화한 결과입니다

그래프에서 볼 수 있듯이, 배치 사이즈가 작을수록 더 높은 최종 정확도에 도달하는 경향을 명확하게 확인할 수 있습니다. 이는 배치 사이즈가 모델의 최적 성능을 이끌어내는 데 매우 중요한 튜닝 파라미터임을 보여줍니다.



혼동행렬 핵심 구조

혼동행렬(Confusion Matrix)은 분류 모델의 예측 결과를 실제 정답과 비교하여 표로 정리한 도구

4가지 예측 결과

TP (True Positive): 실제 긍정을 긍정으로 정확히 예측

TN (True Negative): 실제 부정을 부정으로 정확히 예측

FP (False Positive): 실제 부정을 긍정으로 잘못 예측 (1종 오류)

FN (False Negative): 실제 긍정을 부정으로 잘못 예측 (2종 오류)

화재 경보기

TP: 화재 발생 → 경보 울림 (정확!)

FN: 화재 발생 → 경보 안 울림 (위험한 상황)

TN: 화재 없음 → 경보 안 울림 (정상)

FP: 화재 없음 → 경보 울림 (거짓 경보)

질병 진단

TP: 실제 환자 → 환자로 진단 (정확한 진단)

FN: 실제 환자 → 정상으로 진단 (위험한 오진)

TN: 건강한 사람 → 정상 진단 (올바른 판단)

FP: 건강한 사람 → 환자로 진단 (불필요한 불안)

스팸 메일 필터

TP: 스팸 메일 → 스팸함으로 분류 (성공)

FN: 스팸 메일 → 받은편지함 도착 (놓침)

TN: 정상 메일 → 받은편지함 도착 (정상)

FP: 정상 메일 → 스팸함 분류 (중요 메일 놓칠 위험)

▶ 오류 유형별 대응 전략



FP가 높을 때 (오탐 많음)

원인: 임계값이 너무 낮거나 모델이 과도하게 민감함

해결책:

1. 임계값 상향 조정 (예: 0.5 $\rightarrow$ 0.7)
2. 정밀도(Precision) 중심 최적화
3. FP 페널티 강화

적용 사례: 스팸 필터(중요 메일 보호), 광고 타겟팅



FN이 높을 때 (놓침 많음)

원인: 임계값이 너무 높거나 모델이 지나치게 보수적

해결책:

1. 임계값 하향 조정 (예: 0.5 $\rightarrow$ 0.3)
2. 재현율(Recall) 중심 최적화
3. FN 페널티 강화

적용 사례: 암 진단(환자 놓치면 위험), 사기 탐지



균형 잡힌 성능 필요

목표: FP와 FN 모두 최소화

해결책:

1. F1-Score 최적화
2. ROC 곡선으로 최적 임계값 탐색
3. 교차 검증으로 안정성 확보

적용 사례: 일반 분류 문제, 비용이 비슷한 경우

3개 클래스(개, 고양이, 토끼) 분류 모델 성능 분석(각 동물 50마리씩 총 150마리를 예측한 결과)

실제 \ 예측	개	고양이	토끼
개 (50)	45	3	2
고양이 (50)	2	40	8
토끼 (50)	3	7	40

분석 결과

- 총 정확도: $125/150 = 83.3\%$
- 가장 많은 오류: 고양이를 토끼로 잘못 분류 (8개)
- 개선 방향: 고양이-토끼 구별 특징 강화 필요

대각선 값(45, 40, 40)은 올바른 예측을 나타내며, 비대각선 값들은 잘못된 예측입니다. 고양이와 토끼를 구별하는 특징(털 패턴, 귀 모양 등)을 더 학습시켜야 합니다.

정밀도 (Precision)

정의: 긍정으로 예측한 것 중 실제 긍정의 비율

공식: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

질문: "양성이라고 한 예측이 얼마나 정확한가?"

비유: 명중률 - 10발 쏘서 8발 명중하면 정밀도 0.8

중요한 상황:

- 스팸 필터: 정상 메일 보호
- 제품 추천: 신뢰도 유지
- 법률 판단: 무고한 사람 보호

재현율 (Recall)

정의: 실제 긍정 중 올바르게 예측한 비율

공식: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

질문: "실제 양성을 얼마나 찾아냈는가?"

비유: 완전성 - 10개 목표물 중 8개 찾으면 재현율 0.8

중요한 상황:

- 암 진단: 환자 놓치면 안 됨
- 사기 탐지: 손실 방지
- 테러 탐지: 위험 요소 파악



임계값 0.9 (엄격)

확실한 것만 긍정으로 예측

결과:

- Precision ↑ (정확도 높음)
- Recall ↓ (많이 놓침)



임계값 0.5 (균형)

표준 기준으로 판단

결과:

- Precision = 중간
- Recall = 중간



임계값 0.3 (관대)

의심스러우면 긍정으로 예측

결과:

- Precision ↓ (오탐 많음)
- Recall ↑ (거의 안 놓침)

- ❏ 실무 팁: 임계값을 조정하여 비즈니스 요구사항에 맞는 균형점을 찾아야 합니다. 의료 진단처럼 놓치면 안 되는 경우는 낮은 임계값(높은 Recall), 스팸 필터처럼 오탐을 줄여야 하는 경우는 높은 임계값(높은 Precision)을 선택합니다.

F1 SCORE 정의

F1 점수는 정밀도와 재현율의 **조화평균**으로, 두 지표의 균형을 평가합니다.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

왜 조화평균일까?

일반 평균의 문제점:

- Precision = 1.0, Recall = 0.1
- 평균 = 0.55 (중간처럼 보임)
- 불균형을 숨기는 문제

조화평균의 장점:

- 같은 경우 F1 = 0.18
- 하나라도 낮으면 F1도 낮음
- 균형을 강제함

다중 클래스 분류에서는 각 클래스의 성능을 어떻게 평균낼지가 중요합니다.
세 가지 방식은 각기 다른 목적에 적합합니다.



매크로 평균 (Macro)

계산: 각 클래스 지표를 단순 평균

특징: 모든 클래스를 동등하게 취급, 소수 클래스 성능도 반영

예시: 클래스 A(100개) F1=0.9, 클래스 B(50개) F1=0.8, 클래스 C(10개) F1=0.3

Macro-F1 = $(0.9 + 0.8 + 0.3) / 3 = 0.67$

사용 시기: 모든 클래스가 동등하게 중요, 소수 클래스 모니터링 필요



마이크로 평균 (Micro)

계산: 모든 클래스의 TP, FP, FN 합산 후 계산

특징: 다수 클래스의 영향이 크고, 전체 정확도와 유사

예시: 클래스 A TP=90, 클래스 B TP=45, 클래스 C TP=3

Micro-Precision = $138 / 148 = 0.93$

사용 시기: 다수 클래스가 더 중요, 전체 샘플 정확도 중시



가중 평균 (Weighted)

계산: 각 클래스의 샘플 수로 가중 평균

특징: 클래스 크기를 고려한 균형잡힌 평가

예시: 클래스별 F1에 샘플 수 가중치 부여

Weighted-F1 = $(n1 \times F1_1 + n2 \times F1_2 + n3 \times F1_3) / \text{총 샘플}$

사용 시기: 현실적인 성능 반영 필요, 균형잡힌 평가

ROC 곡선이란?

ROC (Receiver Operating Characteristic)는 임계값을 변화시키면서 모델의 성능을 시각화한 곡선입니다.

- X축: FPR (False Positive Rate) = $FP / (FP + TN)$
- Y축: TPR (True Positive Rate) = Recall

AUC (Area Under Curve)

ROC 곡선 아래 면적으로 모델 성능을 하나의 숫자로 요약합니다.

1.0: 완벽한 분류기 / 0.9~1.0: 매우 우수 / 0.8~0.9: 우수 / 0.7~0.8: 괜찮음 / 0.5: 동전 던지기 수준

AUC 해석

AUC = 0.9의 의미는 "무작위로 선택한 긍정 샘플과 부정 샘플에서, 모델이 긍정 샘플에 더 높은 점수를 줄 확률이 90%"입니다.

모델의 예측 확률이 실제 확률과 일치하도록 조정하는 과정을 확률 보정이라고 합니다. 특히 의료 진단, 자율주행, 금융 리스크 평가처럼 예측 확률 자체를 의사결정에 사용하는 경우 필수적입니다.

보정이 필요한 이유

문제 상황: 모델이 0.8 확률로 예측한 샘플 100개를 모으면?

- 이상적: 80개가 실제 긍정
- 현실: 60개만 긍정 (과신, Over-confident)

기상 예보 비유하자면,

"비 올 확률 80%"라고 예보했다면 실제로 10번 중 8번은 비가 와야 합니다. 6번만 온다면 확률 보정이 필요합니다.

온도 스케일링(Temperature Scaling)

Softmax에 온도(T) 파라미터를 추가하여 확률 분포를 조정합니다.

- $T = 1$: 원래 확률 (변화 없음)
- $T > 1$: 확률 분포가 평평해짐 (덜 확신)
- $T < 1$: 확률 분포가 날카로워짐 (더 확신)

장점: 간단하고 효과적이며, 단일 파라미터만 조정하면 되고 모델 재학습이 불필요합니다.

클래스 불균형(Class Imbalance)은 실무에서 가장 흔한 문제입니다.

예를 들어 스팸 메일 필터에서 정상 메일 990개, 스팸 10개인 경우

모든 메일을 정상으로 예측해도 99% 정확도를 얻지만 쓸모없는 모델이 됩니다.

이를 해결하기 위한 전략으로 다음 4가지가 있습니다.

1. 가중 손실
2. SMOTE
3. 데이터 증강
4. 앙상블 기법



가중 손실 (Weighted Loss)

원리: 소수 클래스의 손실에 큰 가중치 부여

장점: 구현이 간단하고 학습 과정에서 해결하며 데이터 변경이 불필요합니다.

사용 시기: 기본적인 첫 번째 시도로 적합



데이터 증강 (Augmentation)

원리: 소수 클래스 샘플을 회전, 반전, 노이즈 추가 등으로 변형하여 증강

장점: 실제로 새로운 정보를 생성하며 과적합 방지 효과

사용 시기: 이미지, 텍스트 데이터에 최적



SMOTE

원리: 기존 샘플 사이를 보간하여 새로운 소수 클래스 샘플 생성

장점: 진짜 새로운 샘플을 만들어 과적합을 감소시키고 일반화 성능 향상

사용 시기: 표 데이터에서 효과적



앙상블 기법

원리: 다수 클래스에서 다양한 서브셋 샘플링하여 여러 모델 학습 후 앙상블

장점: 정보 손실 최소화하며 안정적 성능 제공

사용 시기: 최종 프로덕션 모델